

Trust, but Verify: Evaluating LLMs as Cross-Auditors of LLM Generated Android Vulnerability Analysis

Toward Designing a Multi-Agent Framework for Auditing LLM-Guided Android Security Analysis

João R. Cardoso
University of Southern Denmark
MMMI
Odense, Denmark
jocar25@student.sdu.dk

Lily B. Wanscher
University of Southern Denmark
MMMI
Odense, Denmark
liwan21@student.sdu.dk

Denisa A. Rissa
University of Southern Denmark
MMMI
Odense, Denmark
deris22@student.sdu.dk

Yusuf M. Yusuf
University of Southern Denmark
MMMI
Odense, Denmark
yuyus19@student.sdu.dk

Abstract—

I. INTRODUCTION

Hybrid android applications occupy a uniquely challenging position in mobile security. By enabling communication between a WebView and native Java or Kotlin code to the extents of arbitrary code execution, developers gain the ability to use existing web standards to manage and render content, but they also introduce an attack surface that is difficult to reason about for the individual developer, or analyze statically.

The central mechanism is `addJavaScriptInterface`, an Android API that exposes Java objects and their methods directly to JavaScript running inside the WebView. The implications are significant: any JavaScript executing in that context can invoke arbitrary Java methods on the bridge, including operations that touch the file system, send network requests, query databases, or leak device identifiers. When those methods lack proper input validation, origin checking, or access control, the result is a class of vulnerabilities spanning both sides of the language boundary simultaneously, including path traversal, intent redirection, PII exposure, and SQL injection.

Large Language Models present a compelling complement to traditional static analysis tools in this space. Given a structured summary of bridge interfaces alongside recovered JavaScript snippets, an LLM can reason across the language boundary, infer semantic intent from naming conventions, trace plausible data-flow paths, and generate findings that a symbol-level static analyzer would miss. The obstacle is not capability, it is trust. LLMs are non-deterministic and prone to hallucination, producing confident-sounding findings that reference methods which do not exist, invent data-flow paths not present in the evidence, or overstate the severity of issues

whose preconditions are never met. In a security context, this unreliability is not a minor inconvenience but a fundamental barrier to operational use.

This paper presents Trust, but Verify, a multi-agent framework designed to quantify how much an LLM-generated security report can be trusted. The core idea is to separate analysis from verification: one set of LLM agents performs the security analysis, and a second set evaluates each output against the raw evidence, producing structured quality metrics rather than a binary verdict. For each analyzed callsite, the framework computes four metrics covering hallucination frequency, technical accuracy, effect-chain awareness, and attack-surface coverage, which combine into a composite Trustworthiness Score ranging from -1.0 to 1.0.

The remainder of this paper is structured as follows.

A. Contribution

This paper aims to contribute to existing literature by attempting to quantify and present how capable various models are of verifying their own, and others', work within the context of security analysis. One aspect is how good a large language model may be at producing functional code in terms of programmatic proficiency and language awareness, another is how capable the same, or another, model may be at analyzing and catching mistakes and vulnerabilities on their own.

In other words, this paper aims to find and prove how trustworthy certain models are.

B. Concessions

Testing population diversity was limited due to lack of compute and funding. As in, the largest model tested had but

14 billion parameters. This will be further detailed in Testing & Population.

II. STATE OF THE ART

A. Static Analysis of Android Hybrid Applications

The security challenges introduced by Android’s WebView bridge mechanism have attracted considerable attention from the static analysis community. The central difficulty is that hybrid applications combine two execution environments, Java and JavaScript, whose interaction is difficult to reason about from either side alone.

LUDroid, proposed by Tiwari et al. [1], represents one of the most comprehensive large-scale studies of this problem. The framework performs information flow analysis on 7,500 real-world applications from the Google Play Store, extracting data flows from Android to JavaScript through the `addJavascriptInterface` and `loadUrl` APIs. LUDroid decompiles APKs using APKTool and applies backward slicing to identify the Java objects injected into WebView contexts, then categorizes the resulting flows as sensitive or benign based on established Android source-sink definitions. Its findings are significant: 68% of sampled applications used WebView, and among those, 82% of sensitive information flows could reach potentially untrusted JavaScript code. The study also identified 365 applications loading content over unencrypted HTTP, exposing them to man-in-the-middle substitution attacks, and found 653 applications where JavaScript transmitted data back to bridged Android objects, implementing a two-way communication channel that conventional one-directional taint analyses would miss entirely.

LUDroid’s contribution is primarily empirical: it establishes what the WebView bridge is used for in practice and where the dangerous patterns concentrate. Its limitations are equally instructive. The framework does not follow the JavaScript side of the bridge dynamically; code fetched from remote servers or constructed at runtime falls outside its analysis scope. The authors explicitly note that any JavaScript loaded via unencrypted protocols, or injected from third-party libraries, is treated as a black box. This boundary precisely defines the gap that motivates the present work.

iWanDroid [2] addresses a complementary aspect of the same problem. Rather than focusing on information flow statistics, it provides structured extraction of WebView bridge interface definitions from real-world open source applications, drawing primarily from the F-Droid repository. The resulting dataset, which forms the evaluation corpus for this paper, encodes bridge interface declarations, exposed method signatures, and associated metadata in a form suitable for downstream analysis. Like LUDroid, iWanDroid operates statically and therefore cannot observe JavaScript behavior at runtime or reason about code loaded from remote origins.

B. Limitations of Static Approaches and the Role of Semantic Reasoning

Taken together, tools such as LUDroid and iWanDroid demonstrate that the Java side of the WebView bridge is

tractable for static analysis: method signatures, injected class names, and data-flow paths through the Android codebase can be extracted at scale with reasonable precision. What neither approach can provide is semantic interpretation of what those interfaces imply in a concrete attack scenario. A method named `readFile` accepting a path parameter is structurally visible to static analysis but its exploitability under a path traversal attack requires reasoning about naming conventions, parameter semantics, and plausible attacker behavior that falls outside the scope of symbol-level tools.

This gap motivates the use of Large Language Models as an analytical layer on top of statically extracted bridge data. Recent work on LLM-assisted vulnerability detection has shown that frontier models can reason across abstraction boundaries, infer intent from naming conventions, and generate coherent findings from structured program summaries [3]. The central challenge, as this paper addresses, is not whether such reasoning is possible but whether it is reliable enough to be trusted in an operational security context.

C. Multi-Agent Frameworks for LLM Reliability

A recurring finding across LLM-assisted security analysis is that single-model outputs are unreliable in isolation: they produce plausible but incorrect findings, and no internal signal distinguishes a hallucinated result from a well-evidenced one. Multi-agent architectures have emerged as a structural response to this problem, distributing generation and verification across distinct agent roles so that one model’s output becomes another model’s input for scrutiny.

The foundational motivation for this design comes from work on multi-agent debate in general reasoning tasks. Liang et al. [4] identify a *Degeneration-of-Thought* problem in self-reflective LLMs: once a model has committed to a position, iterative self-correction fails to dislodge incorrect conclusions. Their Multi-Agent Debate (MAD) framework addresses this by having multiple agents argue opposing positions under a judge agent, demonstrating that inter-agent disagreement surfaces errors that single-model reflection cannot. Crucially, they also find that using different LLMs as agents introduces fairness problems in adjudication, a concern directly relevant to multi-model auditing designs.

This debate principle has been applied directly to security analysis. GPTLens [5] proposes an adversarial two-stage framework for smart contract vulnerability detection in which an auditor agent generates candidate vulnerabilities broadly, and a critic agent then evaluates each finding for correctness, severity, and exploitability. The separation of generation from discrimination significantly reduces false positives compared to single-stage detection, while preserving recall. However, GPTLens uses the same underlying model for both roles, which limits the independence of the critique: the critic inherits the same biases and blind spots as the auditor. The Trust, but Verify framework extends this principle by decoupling the two roles across different models entirely, allowing the Auditor’s assessment to be genuinely independent of the Inspector’s assumptions.

More recent work reinforces both the promise and the limits of the approach. Sheng et al. [3] identify reliability and verification of model outputs as central open problems in LLM-based security analysis, noting that false positive rates remain high and that incorrect reasoning can accompany correct vulnerability identification. This compounds the challenge: an audit layer must evaluate not only whether a finding is present in the evidence, but whether the reasoning chain connecting evidence to conclusion is sound. The Trustworthiness Score defined in this paper operationalizes exactly that distinction through its separation of hallucination frequency from technical accuracy and effect-chain awareness.

III. THEORETICAL CONCEPTS

A. Hybrid Android Applications and WebView

Hybrid Android applications combine native Android code with web-based content rendered through WebView. In this paper, the important feature of this architecture is not the use of web content itself, but the presence of two interacting execution environments: Java or Kotlin on the Android side and JavaScript inside the WebView. This distinction matters because each environment operates under different security assumptions. Native code may access Android resources such as files, intents, databases, network APIs, and device identifiers, while JavaScript is usually constrained by the web context in which it executes. Security issues arise when mechanisms exist for JavaScript to reach native functionality [6].

B. Java-JavaScript Bridges

A Java-JavaScript bridge is the interface through which JavaScript inside a WebView can invoke native Android methods. Android commonly provides this functionality through `addJavascriptInterface`, which exposes a Java or Kotlin object to the JavaScript environment under a chosen interface name [6].

For this work, the bridge is treated as a security boundary rather than just an implementation mechanism. An exposed method must be evaluated not only by its native implementation, but also by whether it is reachable from JavaScript, whether its inputs can be attacker-controlled, and what downstream effects it can trigger. This bridge-centered view motivates the callsite-level analysis used in the framework.

C. Prompt Engineering

Prompt engineering refers to the practice of designing and refining inputs to language models in order to elicit accurate, relevant, and well-structured outputs. Rather than modifying model weights or fine-tuning on domain-specific data, prompt engineering works within the inference-time interface, shaping model behavior through the construction of the input itself [7].

In the context of this work, prompt engineering is the primary mechanism through which the analysis framework communicates tasks to the language model. Each prompt encodes not only the question to be answered, but also the relevant context, the expected output format, and any constraints the model should respect. Because the model has

no persistent state between invocations, every prompt must be self-contained, carrying all information necessary for the model to produce a useful response.

Several techniques inform the prompts used in this framework. Chain-of-thought prompting encourages the model to produce intermediate reasoning steps before arriving at a conclusion, which has been shown to improve performance on multi-step tasks [?]. Structured output prompting constrains the model to respond in a machine-parseable format such as JSON, allowing downstream components to consume results programmatically without fragile text parsing. Few-shot prompting provides the model with representative examples of input-output pairs, reducing ambiguity and anchoring the model to the expected response style.

Prompt design in security analysis contexts introduces additional considerations. The model must be guided to reason about attacker-controlled inputs, trust boundaries, and reachability conditions, concepts that require precise framing to avoid superficial or incomplete assessments. Poorly constructed prompts risk producing outputs that are syntactically valid but analytically shallow, which motivates treating prompt construction as a first-class engineering concern within the framework.

IV. APPLYING THEORY TO THE DESIGN

A. Prompt Engineering for Inspector and Auditor Agents

Prompt engineering plays a central role in this framework because the models are not asked to produce open-ended security commentary. Instead, each model is assigned a defined role, a fixed evidence context, and a required output structure. This reduces ambiguity and makes outputs easier to compare across different models.

The framework uses a callsite-level unit of analysis rather than an application-level one. Each callsite corresponds to a single WebView bridge row and its associated JavaScript snippets. This narrows the task from broad application auditing to a focused evaluation of one Java-JavaScript bridge interaction.

The Inspector prompt casts the model as a security researcher analyzing a single Android WebView bridge callsite. The provided context includes the application package, Java bridge interface, exposed bridge methods, initiating method, and JavaScript snippets that serve as entry points. The model is then asked to evaluate the callsite using the five criteria shown in Table I.

This prompt design encourages the model to ground its reasoning in the supplied evidence rather than relying on general Android security knowledge. To ensure consistency, each identified vulnerability must include a title, risk level, data-flow path, technical description, and supporting evidence. If no vulnerabilities are found, the model returns a standardized “No vulnerabilities identified” response.

The Auditor prompt serves a different purpose. Rather than generating new findings, it evaluates the Inspector’s output against the same ground-truth context. The Auditor receives the bridge interface, JavaScript snippets, and Inspector findings, then returns a JSON object with four scores: hallucina-

TABLE I
EVALUATION CRITERIA

Category	Description
A. Data Flow & Sink Analysis	Traces variables from JavaScript calls to Java sinks. Flags methods involved in SQL queries, file paths, intent creation, and dangerous execution mechanisms such as <code>Runtime.exec()</code> or reflection.
B. PII & Permission Leakage	Identifies Java methods that return sensitive data, such as <code>getDeviceId()</code> , <code>getAccounts()</code> , <code>getLocation()</code> , and <code>getSimSerialNumber()</code> . Checks whether JavaScript stores PII in <code>localStorage</code> or transmits it externally.
C. Reachability & Attacker Control	Determines which bridge methods are reachable by attacker-controlled JavaScript and how much input to each method is attacker-controlled.
D. Impact Chain & Privilege Escalation	Evaluates the worst-case outcome of invoking exposed methods with arbitrary input and whether multiple calls can be chained to escalate privileges.
E. Protocol & Origin Security	Checks whether content is loaded over <code>http://</code> and whether <code>message.origin</code> is properly validated in event listeners.

tion frequency, technical accuracy, effect-chain awareness, and attack-surface coverage.

This distinction reflects the core principle of *Trust, but Verify*: vulnerability discovery and vulnerability verification should be treated as separate tasks. The Inspector generates possible vulnerability claims, while the Auditor checks whether those claims are supported by the analyzed callsite.

Hallucination is treated as an evidence-grounding failure. A finding is considered hallucinated if it references a method, class, object, code segment, or behavior that does not appear in the provided context. A vulnerability claim is therefore only trustworthy if it can be traced back to the bridge methods, JavaScript snippets, or other callsite evidence.

V. TESTING

A. Metrics

In order to evaluate the abilities of a model for the purposes at hand, a series of metrics were defined. These metrics were defined based on what aspects of model performance were deemed relevant given the context.

- **Hallucination Frequency (HF) [%]:**
How many times did Inspector N hallucinate findings compared to the inspector who hallucinated the most. Requires definition of ‘Hallucination’, i.e. makes up findings or code segment that does not exist in the evaluated case
- **Technical Accuracy (TA) [1..10]:**
Upon any finding, how technically correct is the Inspector in its description of the vulnerability
- **Effect Chain Awareness (ECA) [1..10]:**

For a given identified vulnerability how aware is the agent of the possible chain of effects made possible by this vulnerability

- **Attack Surface Coverage (ASC) [%]:**
How many vulnerabilities did the agent identify compared to researcher evaluation

B. Population

With the testing metrics defined, the testing population was identified. Given limitations as to available compute and funding, larger established models were not available, however any model which could run through Ollama was. This placed a fundamental limit on the diversity achievable in the testing population, especially in terms of model size but specifically in terms of parameter count. Within this limit, the sample was designed to consist of models with different parameter counts, architectures, and providers. The final identified population became:

Smaller models:

- qwen2.5-coder:7b
- deepseek-r1:8b
- starcoder2:7b

Medium models:

- gemma3:12b
- codellama:13b
- phi4:14b

According to K. Sun and R. Wang 2026 [8] the correlation between parameter count and model performance, is rather complex however, thus the upper bound of parameter count among selected models of but 14 billion does not inherently void the testing samples applicability to the generalized population.

C. Storage & Sampling

Before testing started, a JSON structure was designed to keep track of all information involved, generated, and added, for each callsite identified in the IWanDroid dataset. This was done to assure that no information was lost and that the findings would be easily processable when testing concluded.

All in all the dataset generated 144 of such structures, however in respect to time and effort, only 40 entries in total were examined by researchers. These 40 samples were taken as 4 groups of 10 spaced evenly throughout the array. This meant some randomness, but also that there would appear more than one sample for the same app at times - each sample regarding different places in said app.

D. Procedure

The aforementioned JSON structures first contained the models to run as inspectors, the models to run as auditors, the callsite location, package name, and other metadata concerning the application regarded. Later, the structures were appended the inspectors findings in one pass of processing, and then the auditors’ results in another pass. Lastly, each researcher was assigned a group of entries to complete by appending the quantified ground truth in a “researcherConclusion” field by regarding each inspector’s findings, against the auditors conclusion as to the inspector’s performance, against the source code for the application.

E. Post-Processing

Qwen2.5-coder:7b and starcoder2:7b exhibited a behavior of repeating the same series of findings for a given identified callsite many hundreds of times. This led to a series of postprocessing passes being applied to detect repeated findings and replacing it with a corresponding "last N lines repeated M times". Thus making the data easier to work with for the researchers while not losing any information as to the performance of each model.

F. Results

All results were stored in a processable a pre-defined JSON format for easier processing. This segment details the performance differences between the models tested against ground truth as set by the researchers.

VI. DISCUSSION

A. RQ A — Trustworthiness Score

Hallucination Frequency turns out to be statistically independent from the other three metrics ($r = 0.0$ with TA, ECA, and ASC). In practice, this means a model can hallucinate fairly often while still producing accurate findings when it does flag something, and the reverse is equally true. PCA backs this up: PC1 captures 64% of variance and loads almost entirely on TA, ECA, and ASC, while HF lands on PC2. Simply averaging all four metrics equally glosses over this two-dimensional structure.

The most defensible approach to a trustworthiness score is to treat HF on its own terms: one composite for "quality when something is found" (TA + ECA + ASC), with HF handled as a separate penalty. That said, the more pressing issue is inter-auditor agreement. ICC values near zero (-0.10 to 0.11) indicate auditors disagree substantially on individual callsites. This doesn't sink the metrics, but it does mean individual audit scores are noisy. Only aggregated means across multiple auditors are solid enough to cite with confidence.

B. RQ B — Model Benchmarking

Gemma3:12b comes out on top, with phi4:14b, deepseek-r1:8b, and qwen2.5-coder:7b forming a cluster just behind it that is statistically indistinguishable from one another. At the bottom, llama3.2:3b and starcoder2:7b are clearly worse than the leading group (Bonferroni-corrected $p < 0.001$). The notable surprise here is qwen2.5-coder, the only specialist code model in the set, finishing fourth behind two general-purpose models. That result suggests security reasoning across a Java-JS bridge draws more on broad language understanding than on code-specific training. Parameter count also seems to matter: the 12b and 14b models dominate, while the 3b model struggles noticeably. Starcoder2's weak ECA score (mean 4.59 vs. gemma3's 6.85) points to a consistent blind spot in attack chain reasoning. It catches individual issues but tends to miss how they connect downstream.

C. RQ C — Failure Taxonomy

Context window size does not appear to be the main factor driving failures. Context length correlates with performance at $r = 0.13$ across all models, and if anything the relationship nudges slightly positive: richer, longer contexts weakly help rather than hurt. The same holds for bridge complexity. More exposed methods actually correlates with marginally better trustworthiness scores ($p < 0.01$), which suggests that more surface area gives models more to anchor their reasoning to. The real failure modes are elsewhere: hallucination instability and chain reasoning collapse. Llama3.2 hallucinates in 15.9% of audits, nearly three times gemma3's rate, and this doesn't track with context length since hallucination spikes tend to occur in shorter contexts (mean 408 chars vs. 512 for normal audits). The pattern points to low-parameter models confabulating findings when the bridge interface gives them little to work with. Starcoder2 fails differently: its hallucination rate is low, but its effect chain awareness is consistently poor. It surfaces issues at the symptom level but struggles to reason through multi-step exploitation paths. These are meaningfully distinct failure modes and worth keeping separate in the taxonomy: confabulation failure (llama3.2) versus shallow reasoning failure (starcoder2).

VII. CONCLUSION

VIII. PERSPECTIVE

REFERENCES

- [1] A. Tiwari, J. Prakash, S. Groß, and C. Hammer, "LUDroid: A large scale analysis of Android-Web hybridization," in *Proceedings of the IEEE/ACM International Conference on Program Comprehension*, 2019. University of Potsdam.
- [2] A. Tiwari, J. Prakash, and C. Hammer, "iWanDroid: Demand-driven information flow analysis of WebView in Android hybrid apps." The 34th IEEE International Symposium on Software Reliability Engineering (ISSRE), Florence, 2023. Dataset, version 1.0.
- [3] Z. Sheng, Z. Chen, S. Gu, H. Huang, G. Gu, and J. Huang, "LLMs in software security: A survey of vulnerability detection techniques and insights," *ACM Computing Surveys*, 2025. arXiv:2502.07049.
- [4] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, "Encouraging divergent thinking in large language models through multi-agent debate," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 17889–17904, Association for Computational Linguistics, 2024.
- [5] S. Hu, T. Huang, F. Ilhan, S. F. Tekin, and L. Liu, "Large language model-powered smart contract vulnerability detection: New perspectives," in *IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS)*, 2023.
- [6] Android Developers, "Build web apps in webview," 2026. Last updated 2026-05-20 UTC.
- [7] S. Vatsal and H. Dubey, "A survey of prompt engineering methods in large language models for different NLP tasks," 2024.
- [8] K. Sun and R. Wang, "Breaking myths in LLM scaling and emergent abilities with a comprehensive statistical analysis," *Neurocomputing*, vol. 670, p. 132542, 2026.

TABLE II
GROUP MEMBERS AND CONTRIBUTION AREAS

Name	Email	Contribution Area
João R. Cardoso	jocar25@student.sdu.dk	Metric Ideation and Definition, Prompt-Engineering, System Architecture, Manual Result Validation
Lily B. Wanscher	liwan21@student.sdu.dk	Metric Ideation and Definition, System Integrations, Test Suite Execution, Resultset Processing, Manual Result Validation
Denisa A. Rissa	deris22@student.sdu.dk	Metric Ideation and Definition, Manual Result Validation
Yusuf M. Yusuf	yuyus19@student.sdu.dk	LLM Model List, Manual Result Validation

All members mentioned above have contributed in equal importance toward the development of this project.